

PYTHON 3.12+ CHEAT SHEET

Practical syntax, idioms, and standard-library patterns for coding interviews

1. Python Basics

```
# comment; names bind to objects
x, name, active = 42, "Ada", True
a = b = 0                # multiple assignment
missing = None           # absence of a value
```

Type	Typical literal / conversion
int, float, complex	12, 3.5, 2+3j; int("12"), float(x)
bool, NoneType	True, False, None; bool(x)
str, bytes	"text", b"raw"; str(x), bytes(s, "utf-8")
list, tuple	[1,2], (1,2); mutable / immutable sequence
dict, set	{"a":1}, {1,2}; mapping / unique values
<pre>type(x) is int # exact type isinstance(x, (int, float)) # type or subclass a == b # equal values a is b # identical object (use: x is None)</pre>	

Falsy: False, None, numeric zero, and empty strings/containers. Everything else is truthy.

References, copies, and hashability

Names refer to objects; assignment binds another name, not a copy. == compares values; is compares identity; id(obj) identifies an object during its lifetime.

```
a = [1, 2]; b = a
b.append(3)           # a is now [1, 2, 3]
import copy
shallow = a.copy()     # also a[:] or
                       copy.copy(a)
deep = copy.deepcopy(a) # recursively copies
                       nested objects
```

Shallow copies share nested references. Mutable: list, dict, set. Immutable: int, float, bool, str, tuple, frozenset (a tuple can still contain mutable objects).

Hashable keys/elements: dictionaries and sets use hashing. Usually hashable: numbers, strings, frozenset, and tuples of hashable values. Lists, dicts, and sets are unhashable. hash(x) gets a hash value.

```
d = {(1, 2): "valid"}      # tuple key
# {[1, 2]: "invalid"}     # TypeError: unhashable
                           list
```

2. Strings

Strings are immutable Unicode sequences; operations return new strings.

```
s = "Python"
s[0], s[-1]           # ('P', 'n')
s[1:4], s[::-1]       # ('yth', 'nohtyP')
len(s), "th" in s     # (6, True)
```

Expression	Result / purpose
s.split(",")	split into a list
"-".join(parts)	join strings with separator
s.strip()	trim surrounding whitespace
s.replace("a","b")	replace all occurrences
s.find("th")	first index (2), or -1
s.startswith("Py")	prefix test
s.endswith(".py")	suffix test
s.upper(), s.lower()	change case
s.capitalize(), s.title()	first / each word capitalized
s.count("a")	count non-overlapping matches
s.isalpha(), s.isdigit()	letters only / digits only
s.isalnum(), s.isspace()	letters or digits / whitespace only
"x" in s	substring membership

```
name, age = "Alice", 30
f"{name} is {age} years old"
f"{1234.5:,.2f}"           # '1,234.50'
f"{0.875:.1%}"             # '87.5%'
```

Alignment and display: f"{x:>10}" (right), f"{x:<10}" (left), f"{x:^10}" (center), f"{n:08d}" (zero-pad), f"{x=!r}" (debug).

```
message = """A multiline string
may span several lines."""
raw_path = r"C:\new\data"   # backslashes stay
                           literal
```

ord("a") == 97; chr(97) == "a". Convert characters to/from code points for letter arithmetic.

3. Numbers, Math, and Bits

Syntax	Meaning
a+b, a-b, a*b, a/b	arithmetic; / returns float
a//b, a%b, a**b	floor division, remainder, power
round(x, 2), abs(x)	round; magnitude
min(xs), max(xs), sum(xs)	aggregate values
<pre>import math math.sqrt(9); math.ceil(2.1); math.floor(2.9) math.isclose(a, b, rel_tol=1e-9) math.pi, math.e, math.log(x), math.sin(angle)</pre>	

// and % round toward negative infinity: -7 // 2 is -4 and -7 % 2 is 1. divmod(a, b) returns both at once.

Bitwise operators

These act on the binary bits of an int. Python integers are signed and unbounded, so they behave like two's complement with infinite sign extension.

Syntax	Meaning
a & b	AND: 1 where both bits are 1
a b	OR: 1 where either bit is 1
a ^ b	XOR: 1 where the bits differ
~a	NOT: invert every bit; equals -a - 1
a << n	shift left; same as a * 2**n
a >> n	shift right; same as a // 2**n
<pre>x = 0b1010 # 10; also 0o12, 0xA, 1_000 bin(10), hex(255) # ('0b1010', '0xff') int("1010", 2) # 10 10 & 6, 10 6, 10 ^ 6 # (2, 14, 12) ~10, 10 << 1, 10 >> 1 # (-11, 20, 5) (10).bit_length(), (10).bit_count() # (4, 2)</pre>	
mask = 1 << i	# a 1 in position i only
n mask	# set bit i

```
n & ~mask           # clear bit i
n ^ mask           # toggle bit i
(n >> i) & 1        # read bit i -> 0 or 1
n & (n - 1)        # clear the lowest set bit
n & -n             # isolate the lowest set bit
n > 0 and n & (n - 1) == 0 # exact power of two?
```

Bitwise operators bind tighter than comparisons but looser than arithmetic, so `n & 1 == 0` groups as `(n & 1) == 0`. `>>` preserves the sign and there is no unsigned `>>>`; mask with `& 0xFF` to emulate a fixed width. For named flags, prefer `enum.IntFlag` over bare constants.

4. Lists

Mutable ordered sequences; indexing and slicing match strings.

```
items = ["a", "b", "c"]
items[0], items[-1], items[1:]
items[1] = "B"; "a" in items
empty = []; shallow = items.copy() # see copying
in Basics
```

Mutation	Effect
<code>xs.append(x)</code>	add one item at end
<code>xs.extend(more)</code>	append all items
<code>xs.insert(i,x)</code>	insert before index <code>i</code>
<code>xs.remove(x)</code>	delete first <code>x</code> ; raises if absent
<code>x = xs.pop(i)</code>	delete and return item (last by default)
<code>xs.clear()</code>	remove every item
<code>xs.reverse()</code>	reverse in place
<code>xs.sort(key=..., reverse=True)</code>	sort in place

```
nums = [4, 1, 3, 2]
ordered = sorted(nums) # new list
nums.sort(reverse=True) # mutate nums
```

Useful slices: `xs[:n]` first `n`; `xs[n:]` rest; `xs[::2]` every other; `xs[::-1]` reversed copy; `del xs[i:j]` delete range.

5. Tuples and Sets

Tuples

Immutable sequences are ideal for fixed records and unpacking. A one-item tuple needs a comma.

```
point = (10, 20); single = (10,)
x, y = point
first, *middle, last = values
def bounds(xs): return min(xs), max(xs)
```

Sets

Mutable collections of unique hashable values; no positional indexing.

```
a, b = {1, 2, 3}, {3, 4}
a.add(5); a.discard(9) # no error if absent
a.remove(2)           # KeyError if absent
a | b, a & b, a - b     # union, intersection,
difference
```

`x in a` tests membership; `set(items)` removes duplicates (order not preserved). Use `frozenset(...)` for an immutable, hashable set.

6. Dictionaries

Mappings preserve insertion order and require hashable keys.

```
user = {"name": "Alice", "age": 30}
name = user["name"] # KeyError if absent
age = user.get("age", 0) # safe default
user["active"] = True # add / update
user.update(age=31, city="Paris")
```

Expression	Purpose
<code>k in d</code>	key membership
<code>d.keys(), d.values()</code>	dynamic views
<code>d.items()</code>	(key, value) pairs
<code>d.pop(k, default)</code>	remove key and return value
<code>del d[k]</code>	remove key; raises if absent
<code>d.setdefault(k, v)</code>	get existing or insert default
<code>d1 d2</code>	new merged dict; right side wins

```
for key, value in user.items():
    print(key, value)
lengths = {word: len(word) for word in words}
merged = defaults | overrides
```

Counting and grouped values

```
from collections import Counter, defaultdict
counts = Counter(words) #
counts.most_common(3)
groups = defaultdict(list)
for item in rows: groups[item.key].append(item)
```

7. Conditions

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
```

Comparisons: `==` `!=` `<` `<=` `>` `>=`; logic: `and` `or` `not`. Comparisons chain naturally: `0 <= x < 10`. Short-circuit logic can supply defaults: `name = supplied or "Guest"`.

```
status = "adult" if age >= 18 else "minor"
if ready and not cancelled:
    run()
```

8. Loops

```
for item in items:
    process(item)
for i in range(0, 10, 2): # 0, 2, ..., 8
    print(i)

for i, value in enumerate(items, start=1):
    print(i, value)
for name, score in zip(names, scores, strict=True):
    print(name, score)

for key, value in data.items():
    if value is None: continue
    if key == target: break
else:
    print("no break / not found")

attempts = 3
while attempts > 0:
    if try_once(): break
    attempts -= 1
```

`break` exits the nearest loop; `continue` skips to its next iteration. Loop else runs only when no `break` occurs.

9. Functions

```
def greet(name: str, greeting: str = "Hello") -> str:
    """Return a short greeting."""
    return f"{greeting}, {name}!"

greet("Ada", greeting="Hi")
```

Parameters may be positional or keyword. / marks positional-only; * marks following parameters keyword-only.

```
def clamp(x, /, low=0, high=100, *, strict=False):
    if strict and not low <= x <= high:
        raise ValueError(x)
    return max(low, min(x, high))

def collect(*args, **kwargs):
    return args, kwargs          # tuple, dict

nums = [3, 7]
range(*nums)                   # unpack positional args
opts = {"sep": " - ", "end": "!\n"}
print("a", "b", **opts)       # unpack keyword args
```

Lambdas are short expressions, best used as keys/callbacks: `sorted(users, key=lambda u: u["age"])`.

Mutable default trap: defaults are created once.

```
def add(x, items=None):
    items = [] if items is None else items
    items.append(x)
    return items
```

Scope: LEGB, closures, decorators

Name lookup checks **L**ocal, **E**nclosing, **G**lobal, then **B**uilt-in. `global` rebinds a module name; `nonlocal` rebinds a name in the nearest enclosing function scope.

```
x = 10
def change_global():
    global x
    x += 1

def outer():
    y = 20
    def inner():
        nonlocal y
        y += 1
    inner()
    return y
```

Closures capture a variable, not its value when the function is created (late binding):

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)
[f() for f in funcs]          # [2, 2, 2]
# Fix inside loop: funcs.append(lambda i=i: i)

from functools import wraps
def decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print("before")
        return func(*args, **kwargs)
    return wrapper

@decorator
def hello(): print("hello")
```

@decorator above `def f(): ...` means `f = decorator(f)`. `wraps` preserves the wrapped function's metadata.

10. Comprehensions and Generators

Result	Pattern
list	<code>[f(x) for x in xs if test(x)]</code>
set	<code>{f(x) for x in xs}</code>
dict	<code>{k: f(k) for k in keys}</code>
generator	<code>(f(x) for x in xs)</code> (lazy)

```
even_squares = [x*x for x in nums if x % 2 == 0]
by_id = {u.id: u for u in users}
total = sum(x*x for x in nums)  # no intermediate list
flat = [x for row in matrix for x in row]
```

Prefer a normal loop when the expression needs side effects or becomes hard to scan.

11. Most Useful Built-ins

Purpose / calls	Use
Inspect: <code>type(x)</code> , <code>isinstance(x,T)</code>	exact type / subclass-aware test
<code>id(x)</code> , <code>hash(x)</code> , <code>len(x)</code>	identity / hash / size
Convert: <code>int(x)</code> , <code>float(x)</code>	number conversions
<code>str(x)</code> , <code>bool(x)</code>	text / truth conversion
<code>list(x)</code> , <code>tuple(x)</code>	sequence conversion
<code>set(x)</code> , <code>dict(x)</code>	unique values / mapping
<code>ord(c)</code> , <code>chr(n)</code>	character / code point
Iterate: <code>range(n)</code> , <code>iter(x)</code>	lazy range / iterator
<code>next(it, default)</code>	next value or default
<code>enumerate(xs)</code> , <code>zip(a,b)</code>	index pairs / paired items
<code>reversed(xs)</code> , <code>sorted(xs)</code>	reverse iterator / sorted list
Aggregate: <code>any(xs)</code> , <code>all(xs)</code>	some / every item truthy
<code>sum(xs)</code> , <code>min(xs)</code> , <code>max(xs)</code>	total / extrema; extrema take <code>key=</code>
<code>divmod(a,b)</code>	quotient and remainder

```
oldest = max(users, key=lambda u: u.age)
has_errors = any(row.error for row in rows)
all_valid = all(validate(x) for x in items)
first = next((x for x in xs if test(x)), None)
```

`map(f, xs)` and `filter(test, xs)` return iterators; comprehensions are usually clearer.

12. Exceptions

Catch only errors you can handle; prefer specific exception types.

```
try:
    value = int(text)
except ValueError as exc:
    value = 0
else:
    use(value)          # conversion succeeded
finally:
    cleanup()           # always runs

if amount < 0:
    raise ValueError("amount must be nonnegative")
```

Multiple types: `except (TypeError, ValueError):`. Re-raise the current exception with bare `raise`; preserve cause with `raise NewError(...) from exc`.

13. Files and pathlib

Context managers close files even when an error occurs. Specify text encoding.

```
with open("data.txt", "r", encoding="utf-8") as f:
    text = f.read()
with open("out.txt", "w", encoding="utf-8") as f:
    f.write(text)
```

"a" appends; "b" selects binary data (for example "rb"). Iterate lines without loading the whole file:

```
with open("data.txt", encoding="utf-8") as f:
    for line in f:
        process(line.rstrip("\n"))
```

Custom context managers

with calls `__enter__()` on entry and `__exit__()` on exit, even after an exception. Return self from `__enter__` for the as target.

```
class Resource:
    def __enter__(self):
        print("acquire")
        return self
    def __exit__(self, exc_type, exc_value,
        traceback):
        print("release")
```

```
with Resource() as r:
    pass
```

Prefer pathlib for paths

```
from pathlib import Path
p = Path("data") / "input.txt"
text = p.read_text(encoding="utf-8")
Path("out.txt").write_text("Hello",
    encoding="utf-8")
```

```
p.exists(); p.is_file(); p.name; p.suffix; p.parent
p.mkdir(parents=True, exist_ok=True)
for file in Path(".").glob("*.py"):
    print(file)
```

`p.rglob("*.csv")` searches recursively. `p.rename(new)` moves a file; `p.unlink(missing_ok=True)` deletes it if present.

14. Modules and Environments

```
import statistics
import numpy as np
from pathlib import Path
from package.module import useful_name
```

Imports run module top-level code once and bind names. Put script entry behavior behind a guard:

```
def main() -> None:
    ...

if __name__ == "__main__":
    main()
```

Packages and `__init__.py`

A directory with `__init__.py` is a regular import package; the file runs on first import and can re-export names. Namespace packages may omit it.

```
# app/__init__.py
from .utils import helper
# elsewhere
from app import helper
```

Configuration with `.env`

`.env` holds KEY=value settings; Python does not load it automatically. Use third-party `python-dotenv`:

```
import os
from dotenv import load_dotenv
load_dotenv()
api_url = os.getenv("API_URL")
```

Existing environment values take precedence by default. Keep secret `.env` files out of version control; share a non-secret `.env.example` template.

Virtual environments

`venv` isolates each project's interpreter and installed packages. Run the commands for your platform:

```
python3 -m venv .venv
py -m venv .venv
source .venv/bin/activate
.venv\Scripts\Activate.ps1
python -m pip install requests python-dotenv
deactivate
```

Exclude `.venv` from version control; activate it before installing or running project code.

15. Classes and Data Classes

```
class Account:
    def __init__(self, owner: str, balance=0.0):
        self.owner, self.balance = owner, balance

    def deposit(self, amount: float) -> None:
        self.balance += amount
```

```
class Savings(Account):
    def __init__(self, owner, rate):
        super().__init__(owner)
        self.rate = rate

    @property
    def interest(self): return self.balance * self.rate
```

Properties: getter and setter

`@property` runs a getter on attribute access; `@age.setter` runs a setter on assignment. Store the value under a different name to avoid recursion.

```
class Person:
    def __init__(self, age): self.age = age
    @property
    def age(self): return self._age
    @age.setter
    def age(self, value):
        if value < 0: raise ValueError("age")
        self._age = value
```

```
p = Person(20)
p.age = 21; print(p.age)
```

Is-a vs. has-a

Savings is an Account: inheritance reuses or extends behavior. **Car has an Engine:** composition stores another object as an attribute.

```
class Engine:
    def start(self): ...

class Car:
    def __init__(self):
        self.engine = Engine()

car = Car()
car.engine.start()
```

Polymorphism

Different subclasses can respond to the same method call in their own way:

```
class Animal:
    def speak(self): return "sound"
class Dog(Animal):
    def speak(self): return "woof"

for animal in (Animal(), Dog()):
    print(animal.speak())           # sound, woof
```

Creation, class state, and methods

MyClass() → __new__(cls) → new object → __init__(self).
cls is the class; self is the instance. __new__ creates/returns an instance; __init__ initializes the already-created instance.

```
class Test:
    a = 100                                # class variable
    def __init__(self):
        self.b = 200                      # instance variable

x = Test(); y = Test()
x.c = 300                                # dynamic instance attribute
```

Test.a is shared unless shadowed by x.a = ...; x.b and y.b belong to separate objects. Define __slots__ to restrict allowed instance attributes.

```
class MyClass:
    def method(self): ...                 # receives instance
    @classmethod
    def create(cls): return cls()         # receives class
    @staticmethod
    def helper(x): return x * 2           # receives neither
```

Important dunder methods

Method	Triggered by / role
__new__, __init__	C() creates, then initializes
__str__	print(obj), str(obj)
__repr__	repr(obj), interactive display
__eq__	a == b
__hash__	hash(obj), dict/set lookup
__len__	len(obj)
__getitem__	obj[i]
__setitem__	obj[i] = x
__iter__	iter(obj), for
__next__	next(obj)
__call__	obj()
__enter__, __exit__	with obj as x:

Singleton (interview pattern)

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

a = Singleton(); b = Singleton()
a is b                                # True
```

__new__ returns the same object on later calls; __init__ still runs each time unless guarded. A module often supplies shared state without a Singleton class.

Use data classes for data containers; they generate __init__, equality, and a useful representation.

```
from dataclasses import dataclass, field
```

```
@dataclass(slots=True)
class User:
    name: str
    tags: list[str] = field(default_factory=list)
```

16. Iterators and Generators

An *iterable* produces an iterator with iter(obj); *iterator* returns itself from __iter__ and supplies values with __next__.
Flow: iterable → iter(obj) → iterator → next(it) → value (or StopIteration).

```
it = iter([10, 20])
next(it), next(it)           # 10, 20
next(it, None)               # None (default avoids StopIteration)
```

```
class Counter:
    def __init__(self, n):
        self.n, self.i = n, 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.i >= self.n:
            raise StopIteration
        self.i += 1
        return self.i
```

Generators produce lazy streams and retain local state:

```
def countdown(n):
    while n:
        yield n
        n -= 1
```

```
squares = (x*x for x in range(1_000_000))
```

Generators are single-use; materialize with list(gen) only when needed.

17. Standard Library Essentials

collections

```
from collections import Counter, defaultdict, deque
Counter("banana").most_common(2) # [('a', 3), ('n', 2)]
groups = defaultdict(list)
queue = deque([1, 2]); queue.append(3);
queue.popleft()
```

heapq (min-heap / priority queue)

A heap is an ordinary list kept in heap order, so h[0] is always the smallest item. Push and pop cost $O(\log n)$ and heapify costs $O(n)$; the list as a whole is *not* sorted.

```
import heapq
h = [5, 1, 3]
heapq.heapify(h)           # -> [1, 5, 3], rearranged in place
heapq.heappush(h, 2)
smallest = heapq.heappop(h) # 1; peek instead with h[0]
heapq.heappushpop(h, 7)    # push then pop, in one pass
heapq.heapreplace(h, 7)    # pop then push; heap must be non-empty

heapq.nsmallest(3, xs)     # sorted; cheap for small k
heapq.nlargest(3, words, key=len)
list(heapq.merge(sorted_a, sorted_b)) # lazy merge of sorted inputs
```

Python 3.12–3.13: negate numbers (or a sort key) for a max-heap:

```
maxh = [-x for x in xs]; heapq.heapify(maxh)
largest = -heapq.heappop(maxh)
```

Python 3.14+: heapq also has heapify_max, heappush_max, and heappop_max.

Entries are compared as whole tuples, so insert a unique tie-breaker ahead of any payload that is not orderable:

```
from itertools import count
tie = count()                # unique, increasing
heapq.heappush(pq, (priority, next(tie), task))
priority, _, task = heapq.heappop(pq) # lowest
```

priority first

Use `queue.PriorityQueue` instead when several threads share the queue.

datetime

```
from datetime import datetime, date, timedelta,
    timezone
now = datetime.now(timezone.utc); today =
    date.today()
dt = datetime.strptime("2026-08-08", "%Y-%m-%d")
dt.strftime("%d %b %Y"); tomorrow = today +
    timedelta(days=1)
```

Use `timezone-aware` datetimes for stored/shared timestamps.

json

```
import json
data = json.loads('{"ok": true}')
text = json.dumps(data, indent=2,
    ensure_ascii=False)
data = json.loads(Path("data.json").read_text())
Path("out.json").write_text(json.dumps(data,
    indent=2))
```

random

```
import random
n = random.randint(1, 6)           # inclusive endpoints
item = random.choice(items)
random.shuffle(items)              # in place
```

Use `secrets` rather than `random` for passwords, tokens, or security.

Regular expressions

Use raw strings for patterns; compile a pattern when reusing it often.

```
import re
m = re.search(r"\b\d+\b", text)    # first match /
    None
nums = re.findall(r"\d+", text)    # all matches
clean = re.sub(r"\s+", " ", text).strip()
```

itertools (compact combinatorics)

```
from itertools import chain, product, combinations
list(chain(xs, ys))
product("AB", repeat=2); combinations(items, 2)
```

functools (memoization and helpers)

```
from functools import lru_cache, cache, wraps,
    partial, reduce

@cache                                # or @lru_cache(None)
def dp(i):
    ...                                # recursive subproblem
```

Memoization turns repeated recursive subproblems into cached lookups; use hashable arguments. `wraps` preserves decorator metadata; `partial(f, x)` fixes arguments; `reduce(f, xs)` folds a sequence.

bisect (sorted-list search)

```
import bisect
nums = [1, 3, 5, 7]
bisect.bisect_left(nums, 5)         # 2: before equals
bisect.bisect_right(nums, 5)        # 3: after equals
bisect.insort(nums, 4)               # keep list sorted
```

Search position: $O(\log n)$; inserting into a list still costs $O(n)$.

18. Working with Data and APIs

Web communication

Client → HTTP request (method, URL, headers, optional body) → server → response (status, headers, body). GET reads; POST submits. Status: 2xx success, 4xx client error, 5xx server error.

HTTP requests and JSON

`requests` is a third-party package; `json` encodes a request body. The `json.loads/dumps` examples above use the standard library.

```
import requests
url = "https://api.example.com/items"
params = {"q": "py"}
r = requests.get(url, params=params, timeout=5)
r.raise_for_status()                # fail on 4xx / 5xx
items = r.json()                    # decode JSON
payload = {"name": "Ada"}
r = requests.post(url, json=payload, timeout=5)
r.raise_for_status()
```

Network errors and resilience

Exception classes below are in `requests.exceptions`.

Exception	Meaning
Timeout	response took too long
ConnectionError	connection / DNS failure
HTTPError	bad status via <code>raise_for_status()</code>
JSONDecodeError	invalid JSON response

Set a timeout. Retry idempotent requests after timeouts, 429, or transient 5xx with bounded backoff; respect `Retry-After`. Do not blindly retry POST, which may duplicate a write.

Storing data with SQLite

```
import sqlite3
con = sqlite3.connect("app.db")
schema = "CREATE TABLE IF NOT EXISTS t(name)"
try:
    with con:                                # commit or rollback
        con.execute(schema)
        sql = "INSERT INTO t VALUES (?)"
        con.execute(sql, ("Ada",))
        cur = con.execute("SELECT name FROM t")
        row = cur.fetchone()
finally:
    con.close()
```

Use `?` placeholders for data values, never string formatting in SQL. `fetchone()` returns one row or `None`; `fetchall()` returns all rows. `with con` does not close it.

19. Common Python Patterns

```
a, b = b, a                                # swap
unique = list(dict.fromkeys(items))        # dedupe, keep
    order
```

Stack and queue

```
stack = []
stack.append(x); x = stack.pop()           # LIFO

from collections import deque
q = deque()
q.append(x); x = q.popleft()               # FIFO / BFS
```

Operation	Cost
<code>list.append(x), list.pop()</code>	$O(1)$ amortized / $O(1)$
<code>list.pop(0)</code>	$O(n)$
<code>deque.append(x), appendleft(x)</code>	$O(1)$
<code>deque.pop(), popleft()</code>	$O(1)$
<code>x in list / x in set/dict</code>	$O(n)$ / average $O(1)$
list indexing / dict lookup	$O(1)$ / average $O(1)$
heap push/pop / sorting	$O(\log n)$ / $O(n \log n)$

Use `deque` for BFS queues, `heapq` for priority queues, `Counter` for frequencies, and `defaultdict(list)` for grouping or adjacency lists. Dicts and sets rely on hashing.

EAFP: handle expected failures

```
try: value = data[key]
except KeyError: value = default
```

20. Common Gotchas

Gotcha	Preferred practice
<code>xs.sort()</code> vs. <code>sorted(xs)</code>	mutates and returns <code>None</code> vs. returns new list
mutable function defaults	use <code>None</code> , then create inside
float equality	use <code>math.isclose(a,b)</code>
mutating during iteration	iterate over <code>xs.copy()</code> or build a new collection

```
# BAD: rows refer to the same inner list
grid = [[0] * 3] * 3
# GOOD: a distinct list per row
grid = [[0] * 3 for _ in range(3)]

# Filtering without modifying the traversed list
items = [x for x in items if keep(x)]
```

Avoid broad `except Exception` unless re-raising or at a deliberate application boundary.